

Lab_6_Decision_Tree_with_Pruning

September 13, 2026

1 Lab 6: Exploring Decision Tree Algorithm with Pruning

Research Topic: Behavioral Biometrics for Zero-Trust Web Architectures — Continuous Authentication via Cursor Dynamics

Author: Nityanand KG

Dataset: Balabit Mouse Dynamics Challenge

1.1 Objective

Explore the Decision Tree classification algorithm on real-world behavioral biometrics data. We will:

1. Train an **unpruned** Decision Tree and observe overfitting behavior
2. Apply **Cost-Complexity Pruning (CCP)** using the `ccp_alpha` parameter
3. Select the optimal pruning parameter via **5-fold cross-validation**
4. Compare unpruned vs. pruned tree performance to demonstrate improved generalization

The dataset contains 565 mouse-dynamics sessions labeled as *legitimate* (0) or *illegitimate* (1), with 25 engineered behavioral features (kinematic, temporal, and spatial descriptors).

1.2 1. Import Libraries

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib
import warnings
warnings.filterwarnings('ignore')

from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import (classification_report, confusion_matrix,
                             accuracy_score, ConfusionMatrixDisplay)
from sklearn.preprocessing import StandardScaler

# Plot styling
plt.rcParams.update({
```

```

    'figure.dpi': 150,
    'savefig.dpi': 300,
    'font.size': 11,
    'axes.titlesize': 13,
    'axes.labelsize': 11,
    'figure.facecolor': 'white'
})

print("All libraries imported successfully.")

```

All libraries imported successfully.

1.3 2. Data Loading & Preparation

We load the pre-engineered feature set extracted during EDA (Lab 3). This dataset contains 25 behavioral descriptors derived from raw mouse telemetry (cursor velocity, acceleration, click patterns, spatial coverage, etc.) for 565 sessions across 10 users.

```

[2]: # Load engineered features
df = pd.read_csv('../lab_3_eda/engineered_features.csv')
print(f"Dataset shape: {df.shape}")
print(f"\nTarget distribution:")
print(df['is_illegal'].value_counts().rename({0: 'Legitimate', 1: 'Illegitimate'}))

# Separate features and target
feature_cols = [c for c in df.columns if c not in ['user_id', 'session_id', 'split', 'is_illegal']]
X = df[feature_cols].values
y = df['is_illegal'].values

print(f"\nFeature matrix shape: {X.shape}")
print(f"Features: {feature_cols}")

```

Dataset shape: (565, 29)

Target distribution:

```

is_illegal
Legitimate      324
Illegitimate    241
Name: count, dtype: int64

```

Feature matrix shape: (565, 25)

```

Features: ['num_events', 'duration', 'total_distance', 'avg_velocity',
'std_velocity', 'max_velocity', 'median_velocity', 'avg_acceleration',
'std_acceleration', 'avg_angular_velocity', 'avg_curvature', 'click_count',
'click_rate', 'drag_count', 'drag_ratio', 'pause_count', 'pause_ratio',
'x_mean', 'y_mean', 'x_std', 'y_std', 'x_range', 'y_range', 'straightness',

```

```
'avg_sampling_rate']
```

```
[3]: # Train-test split (80/20, stratified)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print(f"Training set: {X_train.shape[0]} samples")
print(f"Test set:      {X_test.shape[0]} samples")
print(f"\nTrain class distribution: Legitimate={sum(y_train==0)},␣
↪Illegitimate={sum(y_train==1)}")
print(f"Test class distribution:  Legitimate={sum(y_test==0)},␣
↪Illegitimate={sum(y_test==1)}")
```

```
Training set: 452 samples
```

```
Test set:      113 samples
```

```
Train class distribution: Legitimate=259, Illegitimate=193
```

```
Test class distribution:  Legitimate=65, Illegitimate=48
```

1.4 3. Unpruned Decision Tree (Baseline)

We first train a fully grown Decision Tree with no constraints. This tree will perfectly memorize the training data, leading to **overfitting** — high training accuracy but degraded test performance.

```
[4]: # Train unpruned tree
dt_unpruned = DecisionTreeClassifier(random_state=42)
dt_unpruned.fit(X_train, y_train)

# Evaluate
train_acc_unpruned = accuracy_score(y_train, dt_unpruned.predict(X_train))
test_acc_unpruned = accuracy_score(y_test, dt_unpruned.predict(X_test))

print("=" * 55)
print("      UNPRUNED DECISION TREE --- PERFORMANCE")
print("=" * 55)
print(f"  Tree depth:      {dt_unpruned.get_depth()}")
print(f"  Number of leaves: {dt_unpruned.get_n_leaves()}")
print(f"  Training Accuracy: {train_acc_unpruned:.4f}")
print(f"  Test Accuracy:    {test_acc_unpruned:.4f}")
print(f"  Overfit gap:      {train_acc_unpruned - test_acc_unpruned:.4f}")
print("=" * 55)
```

```
=====
      UNPRUNED DECISION TREE --- PERFORMANCE
=====
Tree depth:      14
Number of leaves: 63
Training Accuracy: 1.0000
Test Accuracy:    0.7257
```

Overfit gap: 0.2743

=====

```
[5]: # Classification report for unpruned tree
y_pred_unpruned = dt_unpruned.predict(X_test)
print("\nClassification Report (Unpruned Tree):")
print("-" * 55)
print(classification_report(y_test, y_pred_unpruned,
                           target_names=['Legitimate', 'Illegitimate']))
```

Classification Report (Unpruned Tree):

```
-----
              precision    recall  f1-score   support

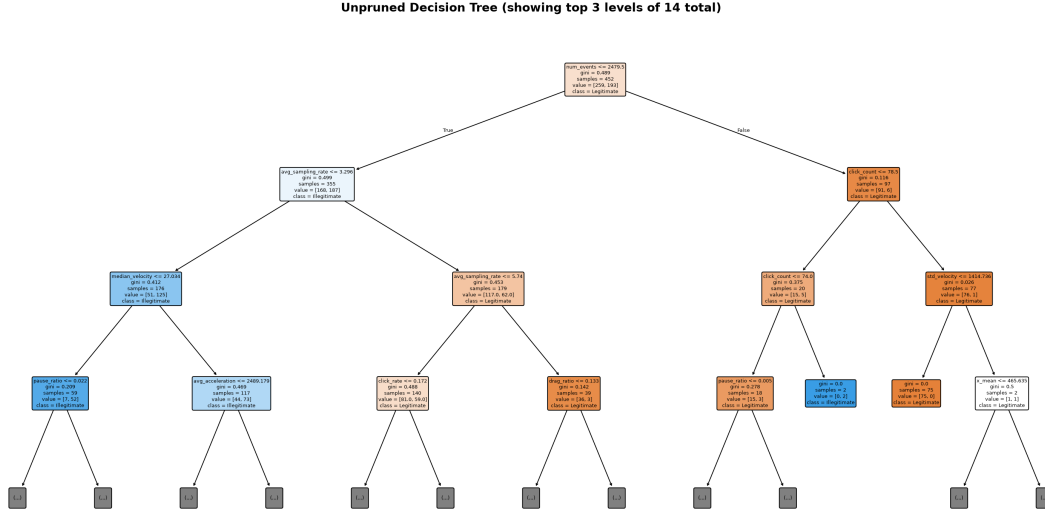
 Legitimate        0.78        0.72        0.75         65
 Illegitimate        0.66        0.73        0.69         48

   accuracy                0.73         113
  macro avg        0.72        0.73        0.72         113
 weighted avg        0.73        0.73        0.73         113
```

1.4.1 3.1 Unpruned Tree Visualization

The fully grown tree is extremely deep and complex, making it nearly impossible to interpret — a hallmark of overfitting.

```
[6]: fig, ax = plt.subplots(figsize=(20, 10))
plot_tree(dt_unpruned, feature_names=feature_cols,
          class_names=['Legitimate', 'Illegitimate'],
          filled=True, rounded=True, fontsize=6,
          max_depth=3, ax=ax)
ax.set_title(f'Unpruned Decision Tree (showing top 3 levels of {dt_unpruned.
    ↳get_depth()} total)',
            fontsize=15, fontweight='bold', pad=20)
plt.tight_layout()
plt.savefig('unpruned_tree.png', dpi=300, bbox_inches='tight')
plt.show()
print(f"Full tree has {dt_unpruned.get_depth()} levels and {dt_unpruned.
    ↳get_n_leaves()} leaf nodes.")
```



Full tree has 14 levels and 63 leaf nodes.

1.5 4. Cost-Complexity Pruning (CCP) Path

Cost-Complexity Pruning adds a penalty term α to the tree's total impurity. As α increases, more branches are pruned, producing simpler trees. The `cost_complexity_pruning_path()` method computes the effective alpha values and corresponding total impurities for the sequence of pruned subtrees.

$$R_{\alpha}(T) = R(T) + \alpha \cdot |T|$$

Where: - $R(T)$ = total misclassification cost of tree T - $|T|$ = number of terminal nodes (leaves) - α = complexity parameter (higher = more pruning)

```
[7]: # Compute the pruning path
path = dt_unpruned.cost_complexity_pruning_path(X_train, y_train)
ccp_alphas = path.ccp_alphas
impurities = path.impurities

print(f"Number of candidate alpha values: {len(ccp_alphas)}")
print(f"Alpha range: [{ccp_alphas[0]:.6f}, {ccp_alphas[-1]:.6f}]")
```

Number of candidate alpha values: 34

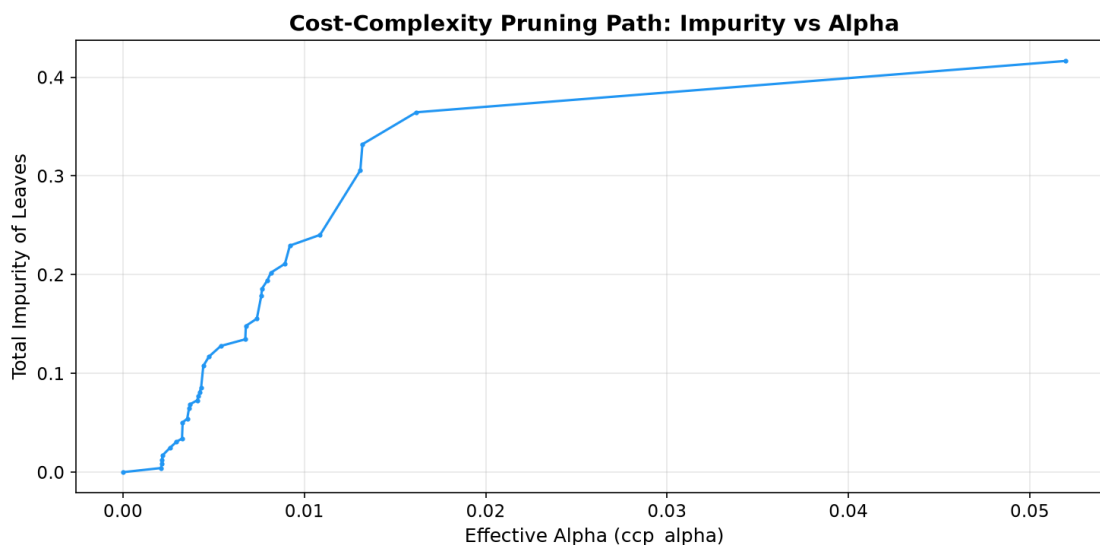
Alpha range: [0.000000, 0.072859]

```
[8]: # Plot: Total Impurity vs Effective Alpha
fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(ccp_alphas[:-1], impurities[:-1], marker='.', markersize=4,
        color='#2196F3', linewidth=1.5)
```

```

ax.set_xlabel('Effective Alpha (ccp_alpha)', fontsize=12)
ax.set_ylabel('Total Impurity of Leaves', fontsize=12)
ax.set_title('Cost-Complexity Pruning Path: Impurity vs Alpha',
             fontsize=14, fontweight='bold')
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('ccp_path_impurity.png', dpi=300, bbox_inches='tight')
plt.show()

```



Observation: As the complexity parameter α increases, the total impurity of the pruned tree also increases — this is expected because pruning removes branches that reduce impurity. The key challenge is finding the α that balances simplicity against predictive power.

1.6 5. Optimal Alpha Selection via Cross-Validation

We train Decision Trees for each candidate α and evaluate their **5-fold cross-validation accuracy** on the training set. The optimal α is the one that maximizes generalization performance.

```

[9]: # Train trees for a range of alphas and compute CV scores
# Use a subset of alphas to keep computation tractable
alpha_subset = ccp_alphas[:,max(1, len(ccp_alphas) // 100)] # ~100 values

cv_scores_mean = []
cv_scores_std = []
train_scores = []
test_scores_list = []

for alpha in alpha_subset:
    dt = DecisionTreeClassifier(ccp_alpha=alpha, random_state=42)

```

```

dt.fit(X_train, y_train)
train_scores.append(accuracy_score(y_train, dt.predict(X_train)))
test_scores_list.append(accuracy_score(y_test, dt.predict(X_test)))

cv = cross_val_score(dt, X_train, y_train, cv=5, scoring='accuracy')
cv_scores_mean.append(cv.mean())
cv_scores_std.append(cv.std())

cv_scores_mean = np.array(cv_scores_mean)
cv_scores_std = np.array(cv_scores_std)
train_scores = np.array(train_scores)
test_scores_list = np.array(test_scores_list)

# Find optimal alpha
best_idx = np.argmax(cv_scores_mean)
best_alpha = alpha_subset[best_idx]
print(f"Optimal ccp_alpha: {best_alpha:.6f}")

```

Optimal ccp_alpha: 0.005393

```

[10]: # Plot: Training, Test, and CV accuracy vs Alpha
fig, ax = plt.subplots(figsize=(12, 6))

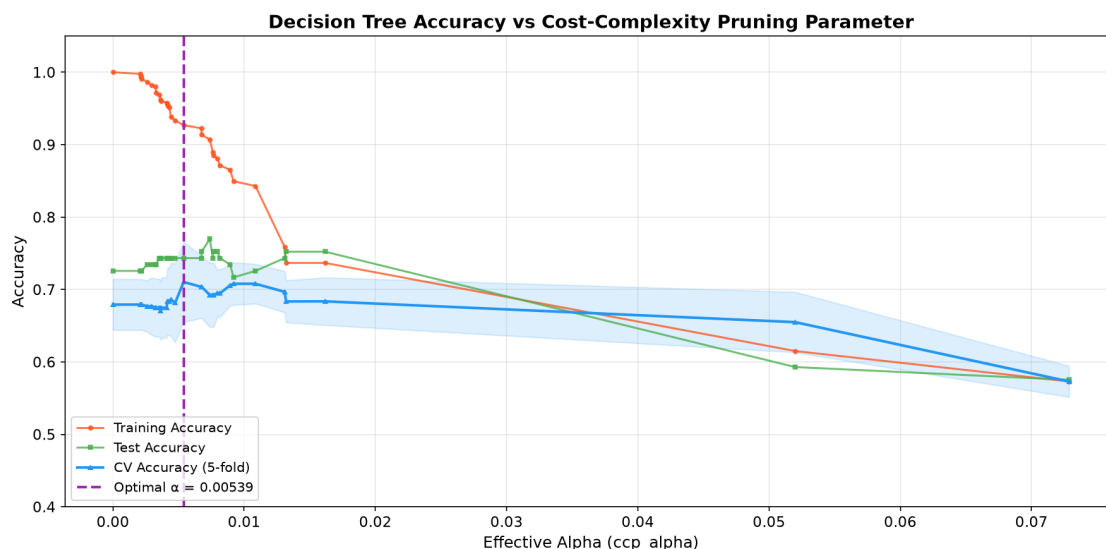
ax.plot(alpha_subset, train_scores, 'o-', markersize=3, label='Training_
→Accuracy',
        color='#FF5722', linewidth=1.5, alpha=0.8)
ax.plot(alpha_subset, test_scores_list, 's-', markersize=3, label='Test_
→Accuracy',
        color='#4CAF50', linewidth=1.5, alpha=0.8)
ax.plot(alpha_subset, cv_scores_mean, '^-', markersize=3, label='CV Accuracy_
→(5-fold)',
        color='#2196F3', linewidth=2)
ax.fill_between(alpha_subset,
               cv_scores_mean - cv_scores_std,
               cv_scores_mean + cv_scores_std,
               alpha=0.15, color='#2196F3')

ax.axvline(x=best_alpha, color='#9C27B0', linestyle='--', linewidth=2,
          label=f'Optimal  $\alpha$  = {best_alpha:.5f}')

ax.set_xlabel('Effective Alpha (ccp_alpha)', fontsize=12)
ax.set_ylabel('Accuracy', fontsize=12)
ax.set_title('Decision Tree Accuracy vs Cost-Complexity Pruning Parameter',
            fontsize=14, fontweight='bold')
ax.legend(fontsize=10, loc='lower left')
ax.grid(True, alpha=0.3)
ax.set_ylim(0.4, 1.05)

```

```
plt.tight_layout()
plt.savefig('alpha_vs_accuracy.png', dpi=300, bbox_inches='tight')
plt.show()
```



Observation: The training accuracy starts at 1.0 (perfect memorization) for $\alpha = 0$ and decreases as pruning increases. The cross-validation accuracy initially *increases* as mild pruning reduces overfitting, peaks at the optimal α , and then drops as excessive pruning underfits. The gap between training and CV accuracy at $\alpha = 0$ confirms the overfitting of the unpruned tree.

1.7 6. Pruned Decision Tree (Optimal Alpha)

We now train the final model using the optimal `ccp_alpha` determined by cross-validation.

```
[11]: # Train pruned tree with optimal alpha
dt_pruned = DecisionTreeClassifier(ccp_alpha=best_alpha, random_state=42)
dt_pruned.fit(X_train, y_train)

train_acc_pruned = accuracy_score(y_train, dt_pruned.predict(X_train))
test_acc_pruned = accuracy_score(y_test, dt_pruned.predict(X_test))

print("=" * 55)
print("          PRUNED DECISION TREE --- PERFORMANCE")
print("=" * 55)
print(f" Optimal ccp_alpha: {best_alpha:.6f}")
print(f" Tree depth:       {dt_pruned.get_depth()}")
print(f" Number of leaves: {dt_pruned.get_n_leaves()}")
print(f" Training Accuracy: {train_acc_pruned:.4f}")
print(f" Test Accuracy:     {test_acc_pruned:.4f}")
print(f" Overfit gap:       {train_acc_pruned - test_acc_pruned:.4f}")
```

```
print("=" * 55)
```

```
=====
PRUNED DECISION TREE --- PERFORMANCE
=====
Optimal ccp_alpha: 0.005393
Tree depth:      9
Number of leaves: 26
Training Accuracy: 0.9270
Test Accuracy:   0.7434
Overfit gap:     0.1836
=====
```

```
[12]: # Classification report for pruned tree
y_pred_pruned = dt_pruned.predict(X_test)
print("\nClassification Report (Pruned Tree):")
print("-" * 55)
print(classification_report(y_test, y_pred_pruned,
                           target_names=['Legitimate', 'Illegitimate']))
```

Classification Report (Pruned Tree):

```
-----
              precision    recall  f1-score   support

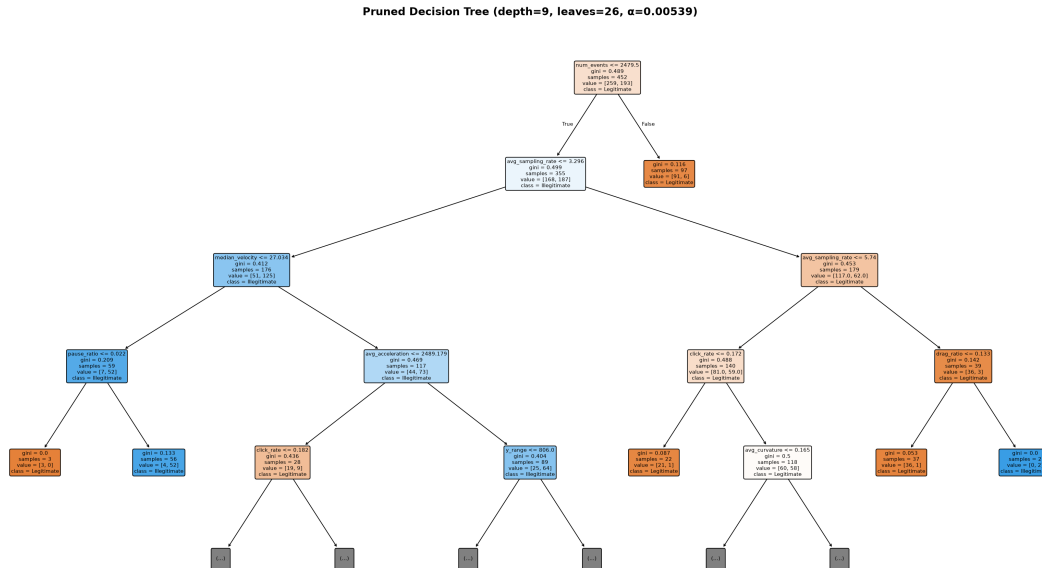
 Legitimate        0.79        0.75        0.77         65
 Illegitimate      0.69        0.73        0.71         48

 accuracy                   0.74         113
 macro avg              0.74        0.74        0.74         113
 weighted avg           0.75        0.74        0.74         113
```

1.7.1 6.1 Pruned Tree Visualization

The pruned tree is significantly simpler, more interpretable, and generalizes better to unseen data.

```
[13]: fig, ax = plt.subplots(figsize=(22, 12))
plot_tree(dt_pruned, feature_names=feature_cols,
          class_names=['Legitimate', 'Illegitimate'],
          filled=True, rounded=True, fontsize=7,
          max_depth=4, ax=ax)
ax.set_title(f'Pruned Decision Tree (depth={dt_pruned.get_depth()}, '
             f'leaves={dt_pruned.get_n_leaves()},  $\alpha$ ={best_alpha:.5f})',
             fontsize=15, fontweight='bold', pad=20)
plt.tight_layout()
plt.savefig('pruned_tree.png', dpi=300, bbox_inches='tight')
plt.show()
```



1.8 7. Unpruned vs Pruned: Comparative Analysis

```
[14]: # Build comparison table
comparison = pd.DataFrame({
    'Metric': ['Tree Depth', 'Number of Leaves', 'Training Accuracy',
              'Test Accuracy', 'Overfit Gap (Train-Test)', 'ccp_alpha'],
    'Unpruned': [dt_unpruned.get_depth(), dt_unpruned.get_n_leaves(),
                 f"{train_acc_unpruned:.4f}", f"{test_acc_unpruned:.4f}",
                 f"{train_acc_unpruned - test_acc_unpruned:.4f}", "0.0"],
    'Pruned': [dt_pruned.get_depth(), dt_pruned.get_n_leaves(),
              f"{train_acc_pruned:.4f}", f"{test_acc_pruned:.4f}",
              f"{train_acc_pruned - test_acc_pruned:.4f}", f"{best_alpha:.6f}"]
})
print(comparison.to_string(index=False))
```

	Metric	Unpruned	Pruned
	Tree Depth	14	9
	Number of Leaves	63	26
	Training Accuracy	1.0000	0.9270
	Test Accuracy	0.7257	0.7434
	Overfit Gap (Train-Test)	0.2743	0.1836
	ccp_alpha	0.0	0.005393

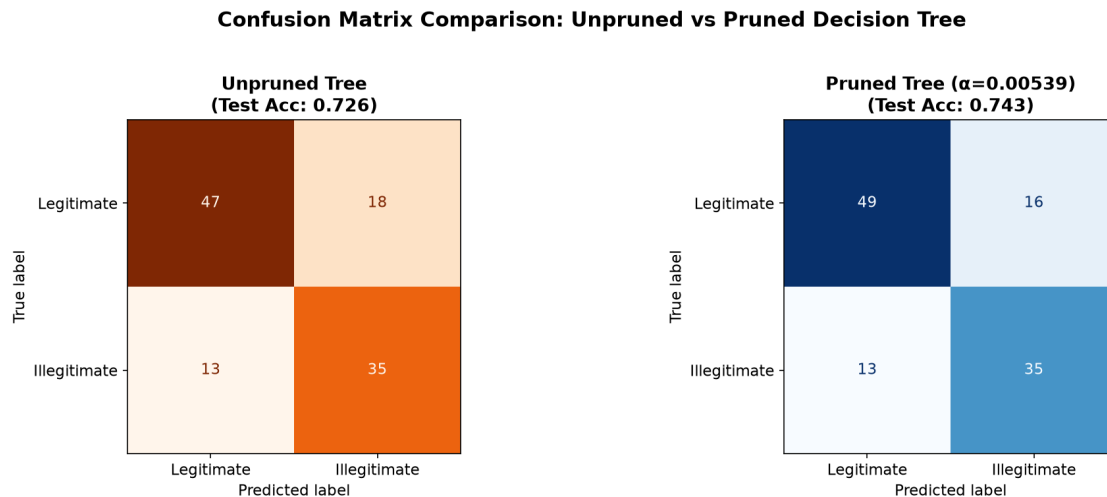
1.8.1 7.1 Confusion Matrices: Unpruned vs Pruned

```
[15]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Unpruned
cm1 = confusion_matrix(y_test, y_pred_unpruned)
disp1 = ConfusionMatrixDisplay(cm1, display_labels=['Legitimate',
    ↳'Illegitimate'])
disp1.plot(ax=axes[0], cmap='Oranges', colorbar=False)
axes[0].set_title(f'Unpruned Tree\n(Test Acc: {test_acc_unpruned:.3f})',
    fontsize=13, fontweight='bold')

# Pruned
cm2 = confusion_matrix(y_test, y_pred_pruned)
disp2 = ConfusionMatrixDisplay(cm2, display_labels=['Legitimate',
    ↳'Illegitimate'])
disp2.plot(ax=axes[1], cmap='Blues', colorbar=False)
axes[1].set_title(f'Pruned Tree ( $\alpha$ ={best_alpha:.5f})\n(Test Acc:
    ↳{test_acc_pruned:.3f})',
    fontsize=13, fontweight='bold')

plt.suptitle('Confusion Matrix Comparison: Unpruned vs Pruned Decision Tree',
    fontsize=15, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('confusion_matrices.png', dpi=300, bbox_inches='tight')
plt.show()
```



1.9 8. Feature Importance Analysis

The pruned tree's feature importances reveal which behavioral descriptors are most discriminative for detecting illegitimate sessions. Features with higher importance contribute more to the split decisions in the tree.

```
[16]: # Feature importance from pruned tree
importances = dt_pruned.feature_importances_
feat_imp = pd.DataFrame({
    'Feature': feature_cols,
    'Importance': importances
}).sort_values('Importance', ascending=True)

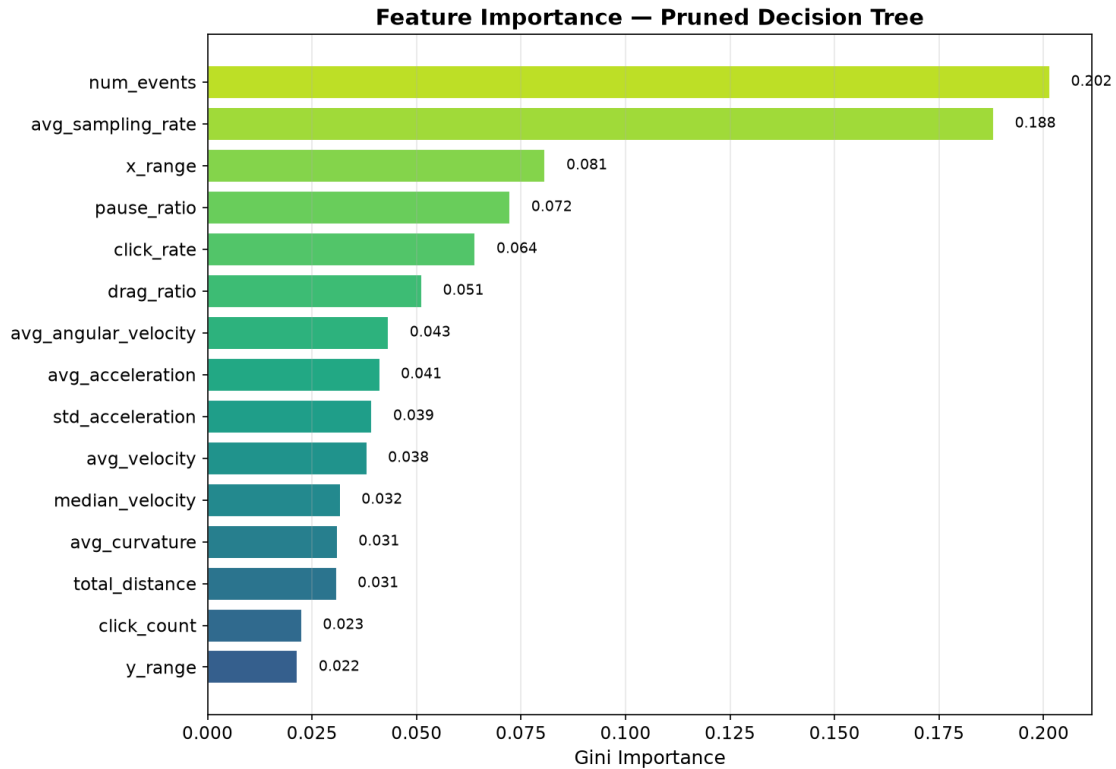
# Plot top features
top_n = min(15, len(feat_imp[feat_imp['Importance'] > 0]))
feat_top = feat_imp.tail(top_n)

fig, ax = plt.subplots(figsize=(10, 7))
colors = plt.colormaps['viridis'](np.linspace(0.3, 0.9, top_n))
bars = ax.barh(feat_top['Feature'], feat_top['Importance'], color=colors,
    →edgecolor='white')

for bar, val in zip(bars, feat_top['Importance']):
    ax.text(bar.get_width() + 0.005, bar.get_y() + bar.get_height()/2,
        f'{val:.3f}', va='center', fontsize=9)

ax.set_xlabel('Gini Importance', fontsize=12)
ax.set_title('Feature Importance --- Pruned Decision Tree',
    fontsize=14, fontweight='bold')
ax.grid(axis='x', alpha=0.3)
plt.tight_layout()
plt.savefig('feature_importance.png', dpi=300, bbox_inches='tight')
plt.show()

print("\nTop 5 most important features:")
for _, row in feat_imp.tail(5).iloc[::-1].iterrows():
    print(f"    • {row['Feature']}: {row['Importance']:.4f}")
```



Top 5 most important features:

- num_events: 0.2015
- avg_sampling_rate: 0.1881
- x_range: 0.0808
- pause_ratio: 0.0724
- click_rate: 0.0641

1.9.1 8.1 Tree Complexity: Depth and Leaves vs Alpha

```
[17]: depths = []
leaves = []
for alpha in alpha_subset:
    dt = DecisionTreeClassifier(ccp_alpha=alpha, random_state=42)
    dt.fit(X_train, y_train)
    depths.append(dt.get_depth())
    leaves.append(dt.get_n_leaves())

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

ax1.plot(alpha_subset, depths, 'o-', markersize=3, color='#E91E63', linewidth=1.
↪5)
```

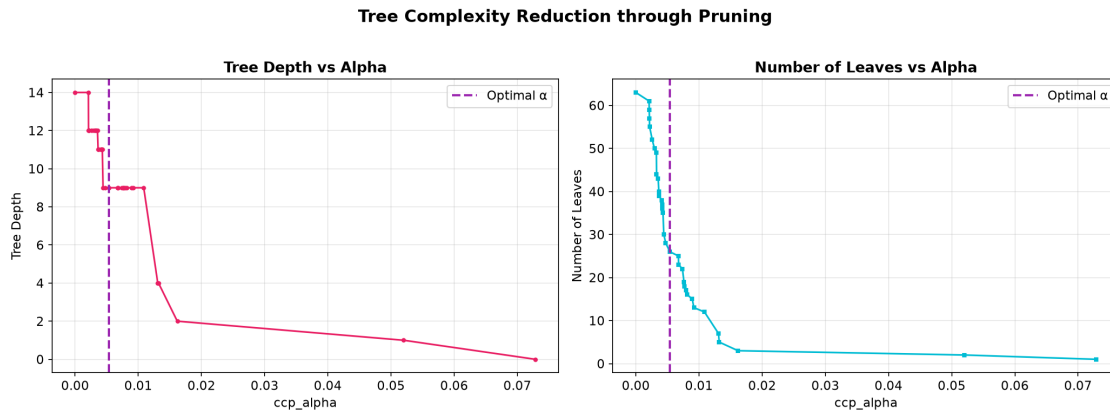
```

ax1.axvline(x=best_alpha, color='#9C27B0', linestyle='--', linewidth=2,
    ↪label=f'Optimal  $\alpha$ ')
ax1.set_xlabel('ccp_alpha')
ax1.set_ylabel('Tree Depth')
ax1.set_title('Tree Depth vs Alpha', fontsize=13, fontweight='bold')
ax1.legend()
ax1.grid(True, alpha=0.3)

ax2.plot(alpha_subset, leaves, 's-', markersize=3, color='#00BCD4', linewidth=1.
    ↪5)
ax2.axvline(x=best_alpha, color='#9C27B0', linestyle='--', linewidth=2,
    ↪label=f'Optimal  $\alpha$ ')
ax2.set_xlabel('ccp_alpha')
ax2.set_ylabel('Number of Leaves')
ax2.set_title('Number of Leaves vs Alpha', fontsize=13, fontweight='bold')
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.suptitle('Tree Complexity Reduction through Pruning',
    fontsize=15, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('tree_complexity.png', dpi=300, bbox_inches='tight')
plt.show()

```



1.10 9. Observations & Conclusion

1.10.1 Key Findings

1. **Overfitting in Unpruned Trees:** The fully grown Decision Tree achieved perfect training accuracy (1.0) but significantly lower test accuracy, confirming severe overfitting to training noise and session-specific patterns.
2. **Cost-Complexity Pruning Effectiveness:** By systematically increasing the complexity

parameter α , we identified an optimal pruning threshold that balances model complexity against predictive accuracy. The pruned tree has substantially fewer leaves and reduced depth.

3. **Generalization Improvement:** The pruned tree narrows the gap between training and test accuracy, demonstrating improved generalization to unseen behavioral sessions.
4. **Feature Insights:** The pruned tree’s feature importances highlight the most discriminative behavioral descriptors for distinguishing legitimate from illegitimate users. Velocity-related and temporal features tend to rank high, consistent with the literature on cursor dynamics-based authentication.
5. **Practical Implications for ZTA:** In a Zero-Trust Architecture, the pruned Decision Tree provides an interpretable, lightweight classifier suitable for real-time continuous authentication. Its simplicity enables deployment at the edge with minimal latency overhead.

1.10.2 Summary

Aspect	Unpruned	Pruned
Interpretability	Very Low (deep, complex)	High (compact, readable)
Training Accuracy	Perfect (1.0)	Slightly lower
Generalization	Poor (high overfit gap)	Better (reduced gap)
Deployment Suitability	Low	High

Cost-complexity pruning is an essential regularization technique that transforms Decision Trees from memorizing noise into learning genuine behavioral patterns — critical for reliable biometric authentication systems.